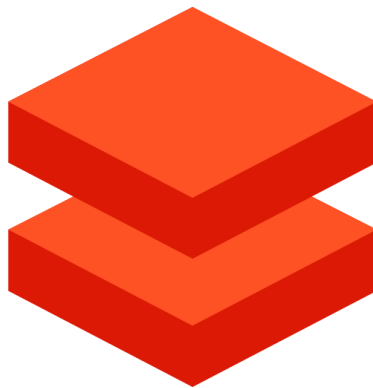
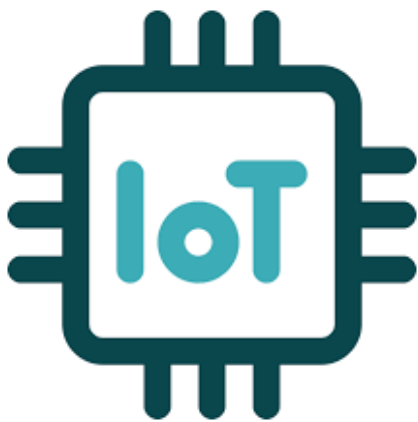


# IoT Real-time Processing with Azure Databricks



## Contents

|                               |    |
|-------------------------------|----|
| Project Overview .....        | 3  |
| 1. Project Description.....   | 3  |
| 2. Project Purpose.....       | 3  |
| 3. Project Objectives .....   | 3  |
| 4. Project Architecture ..... | 4  |
| Technology Stack:.....        | 4  |
| Project Flow: .....           | 5  |
| Technical Documentation ..... | 6  |
| 1. Data Sources .....         | 6  |
| 2. Data Ingestion .....       | 8  |
| i) Azure IoT Hub .....        | 9  |
| ii) Azure Event Hub .....     | 10 |
| 3. Data Transformation .....  | 13 |
| i) Azure Databricks.....      | 13 |
| ii) Code explanation .....    | 15 |
| 4. Data Visualization.....    | 18 |
| 5. Version Control.....       | 19 |
| i) Azure DevOps .....         | 19 |
| References.....               | 20 |

# Project Overview

## 1. Project Description

- ❖ This project focuses on building a data pipeline to ingest and process IoT weather data, storing it in an Azure Databricks data Lakehouse using the Delta format and the Medallion Architecture. Power BI used used to visualize the data for analysis.

## 2. Project Purpose

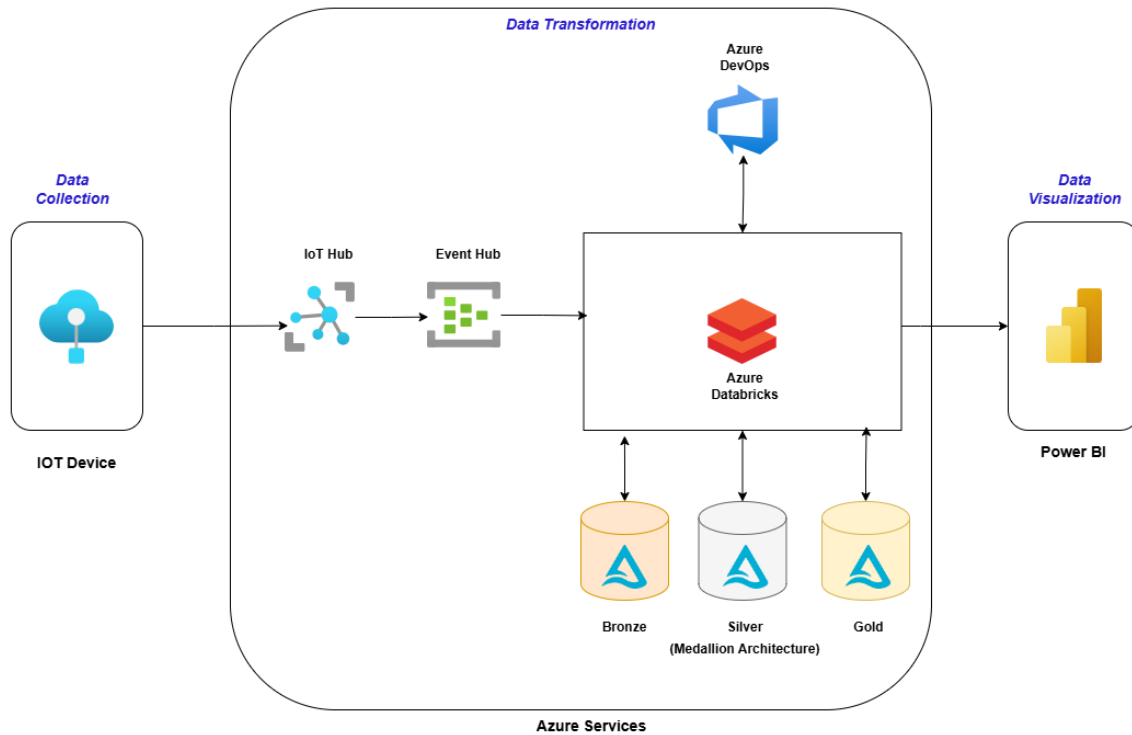
- Develop knowledge in Azure Databricks, Azure DevOps, Data Lake and IoT streaming tools and technologies.
- Improve knowledge in data concepts and tools.
- Demonstrate my interest and enthusiasm for data engineering.

## 3. Project Objectives

- **Data Collection:** Collect weather data from IoT devices over a defined time interval by reading sensor outputs.
- **Data Ingestion:** Establish a network connection to transmit real time IoT weather data for processing.
- **Data Processing/Transformation:** Implement data cleansing, integration, transformation and aggregation processes to convert raw data into meaningful information.
- **Data Storage:** Design and implement a data Lakehouse on Azure Databricks utilising Delta format and the Medallion Architecture (Bronze, Silver, Gold) for weather data storage layer.
- **Data Visualization:** Develop insightful reports and visualizations from the cleansed data using Power BI.
- **Version Control:** Implement a Continuous Integration (CI) pipeline using Azure DevOps and Databricks for version control

## 4. Project Architecture

Architecture Diagram



### Technology Stack:

- Azure IoT Hub
- Azure Event Hub
- Azure Databricks (Delta Lake, PySpark, Python)
- Azure Data Lake Storage Gen2 (ADLS Gen2)
- Azure DevOps
- Power BI
- Git



## Project Flow:

- ❖ Data Sources
  - Simulated IoT devices generating custom weather data (Temperature, Humidity, Wind Speed, Wind Direction, Precipitation, Conditions).
- ❖ Data Pipeline
  - Weather data is published to Azure IoT Hub.
  - Azure Event Hubs subscribes and collects the data, placing it into a consumer group queue.
  - Azure Databricks reads the data into a Spark streaming dataframe and writes it to an Azure Data Lake Storage Gen2 container as Delta files.
- ❖ Data Lakehouse
  - The Medallion Architecture is used to store data in Bronze, Silver and Gold schema layers.
- ❖ Data Visualization
  - Silver layer data is loaded into Power BI.
  - Charts and visuals are used to visualize the data.
- ❖ Continuous Integration (CI)
  - Azure DevOps is used for version control.

# Technical Documentation

## 1. Data Sources

- ❖ For this project, a Raspberry Pi simulator was used as a simulated IoT device, and code was written to generate and transmit customized weather data in given ranges below.

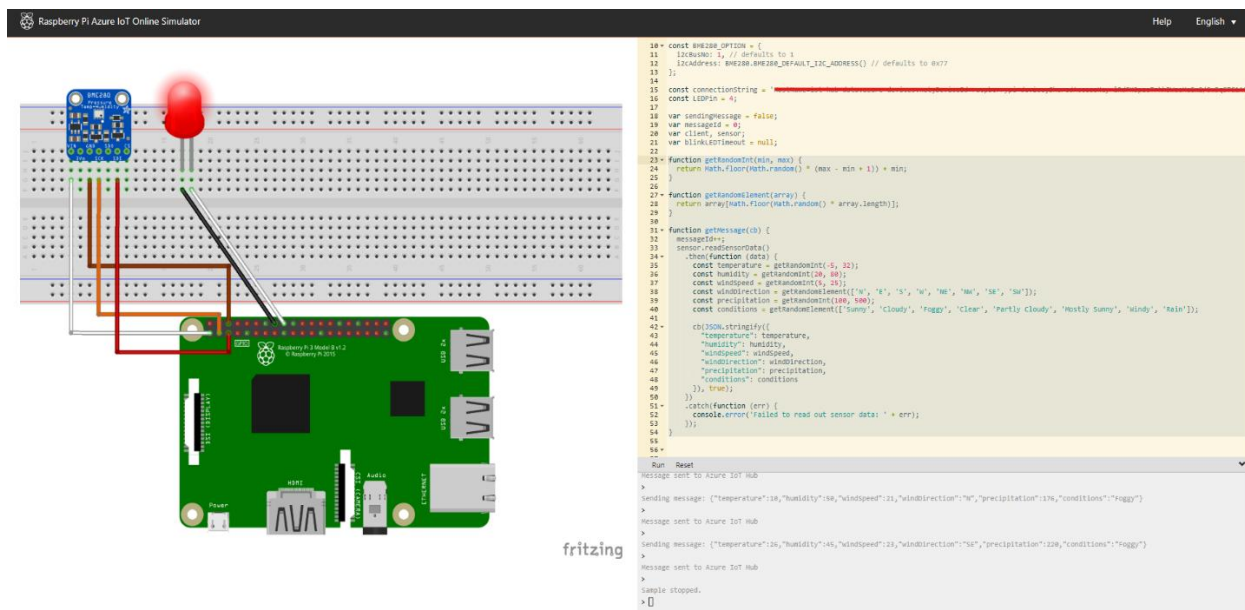
- Temperature : [-5, 32]
- Humidity : [20, 80]
- Wind Speed : [5, 25]
- Wind Direction : ('N', 'E', 'S', 'W', 'NE', 'NW', 'SE', 'SW')
- Precipitation : [100, 500]
- Conditions : ('Sunny', 'Cloudy', 'Foggy', 'Clear', 'Partly Cloudy', 'Mostly Sunny', 'Windy', 'Rain')

- ❖ Sample Payload:

```
{  
  "temperature": 20,  
  "humidity": 60,  
  "windSpeed": 10,  
  "windDirection": "NW",  
  "precipitation": 0,  
  "conditions": "Partly Cloudy"  
}
```

Ex:     {  
          "temperature": 25,  
          "humidity": 60,  
          "windSpeed": 15,  
          "windDirection": "NE",  
          "precipitation": 300,  
          "conditions": "Sunny"  
      }

- ❖ Raspberry Pi Simulator: <https://azure-samples.github.io/raspberry-pi-web-simulator/>



### ❖ Code for the sample payload:

Ex:

```

function getRandomInt(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}

```

```

function getRandomElement(array) {
  return array[Math.floor(Math.random() * array.length)];
}

```

```

function getMessage(cb) {
  messageId++;
  sensor.readSensorData()
    .then(function (data) {
      const temperature = getRandomInt(-5, 32);
      const humidity = getRandomInt(20, 80);
      const windSpeed = getRandomInt(5, 25);
      const windDirection = getRandomElement(['N', 'E', 'S', 'W', 'NE', 'NW', 'SE', 'SW']);
      const precipitation = getRandomInt(100, 500);
      const conditions = getRandomElement(['Sunny', 'Cloudy', 'Foggy', 'Clear', 'Partly Cloudy', 'Mostly Sunny', 'Windy', 'Rain']);
    })
    .catch(function (err) {
      console.error('failed to read out sensor data: ' + err);
    });
}

```

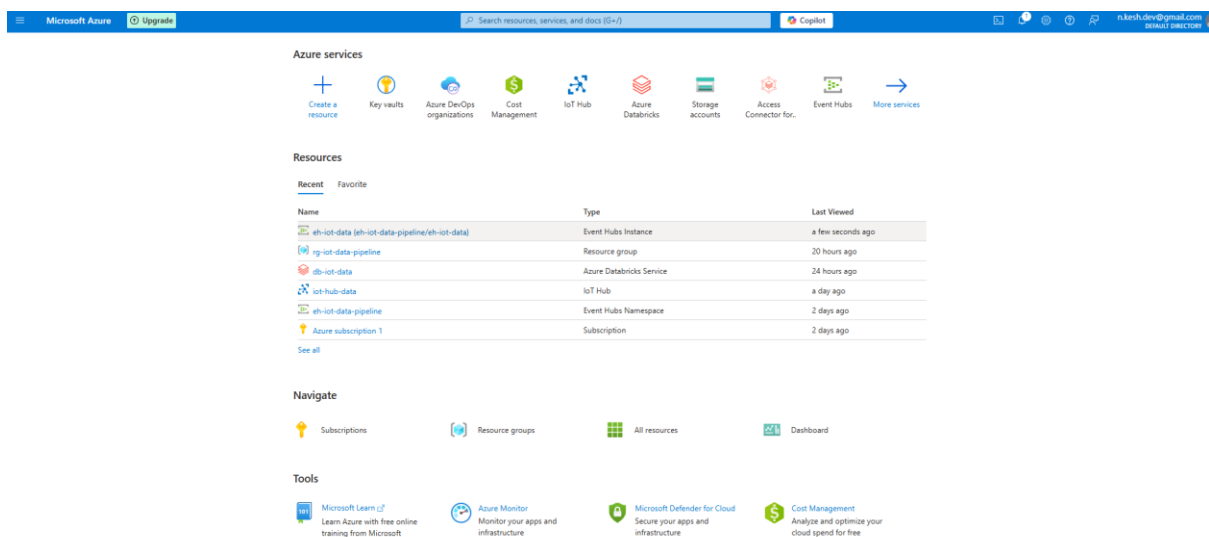
```

    cb(JSON.stringify({
      "temperature": temperature,
      "humidity": humidity,
      "windSpeed": windSpeed,
      "windDirection": windDirection,
      "precipitation": precipitation,
      "conditions": conditions
    })), true);
  })
  .catch(function (err) {
    console.error('Failed to read out sensor data: ' + err);
  });
}

```

## 2. Data Ingestion

- ❖ Initially using a free-tier Azure account with \$200 credit, I built the complete data architecture, including data ingestion, transformation, storage and version control.



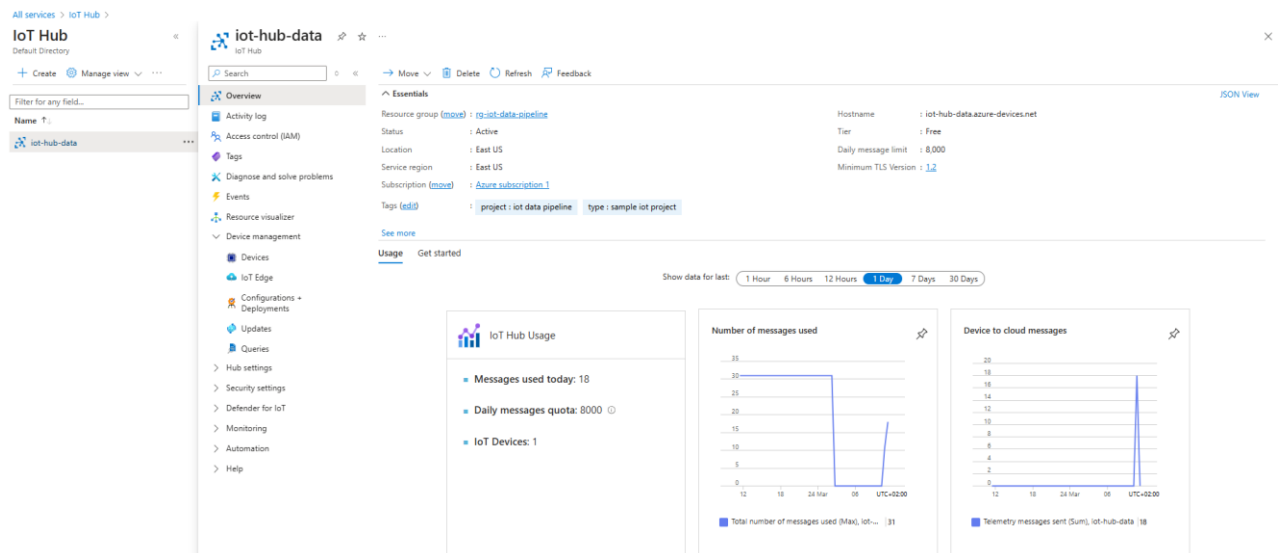


## i) Azure IoT Hub

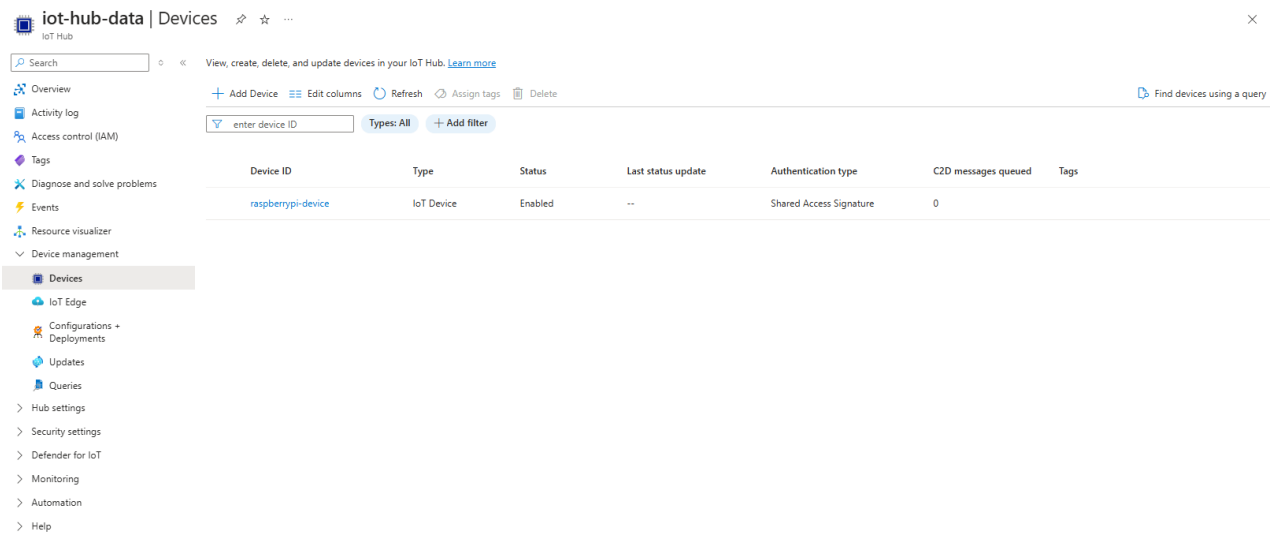
- ❖ Azure IoT Hub was selected as the central communication hub for IoT devices, consolidating data collection into a single point. It provides secure, bidirectional communication between IoT devices and the Azure cloud.
- ❖ Compared to traditional message brokers such as MQTT and NATS, Azure IoT Hub offers improved security, scalability, manageability, cost effectiveness and seamless integration with other Azure services.

### Steps:

1. Create a IoT Hub instance in free tier ( daily message limit 8000)



## 2. Add devices on Device Management



The screenshot shows the Azure IoT Hub 'Devices' management page. The left-hand navigation pane lists various IoT Hub features, with 'Devices' currently selected. The main content area displays a table of registered devices. A search bar and several action buttons (Add Device, Edit columns, Refresh, Assign tags, Delete) are located at the top of the table. A filter is applied, showing 'Types: All'. The table contains one device entry: 'raspberrypi-device', which is an 'IoT Device' with a status of 'Enabled' and no C2D messages queued.

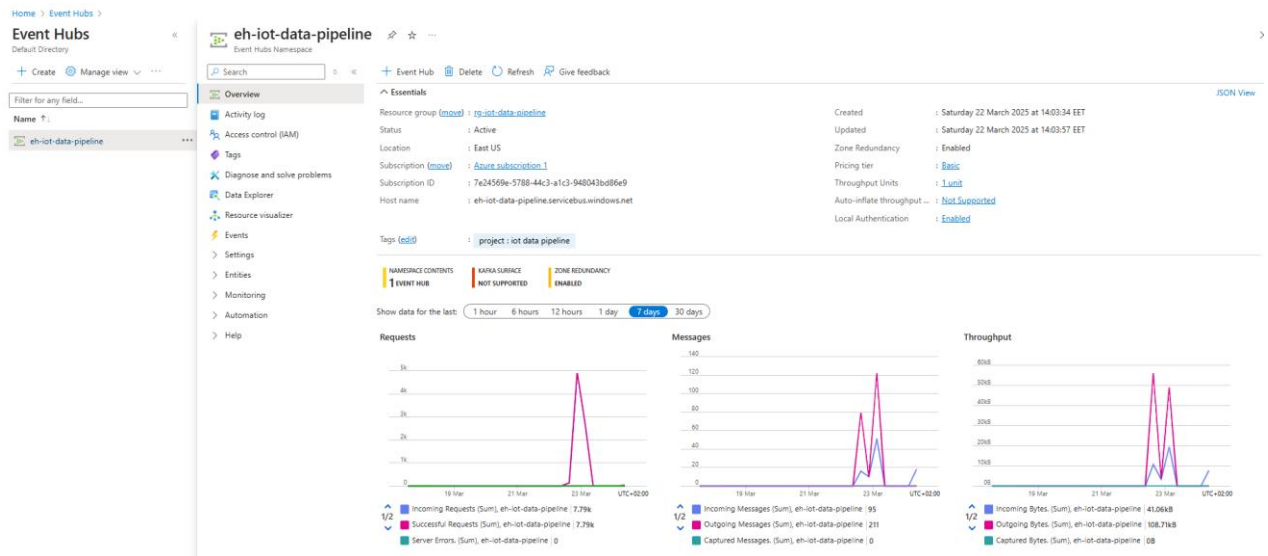
| Device ID          | Type       | Status  | Last status update | Authentication type     | C2D messages queued | Tags |
|--------------------|------------|---------|--------------------|-------------------------|---------------------|------|
| raspberrypi-device | IoT Device | Enabled | --                 | Shared Access Signature | 0                   |      |

### ii) Azure Event Hub

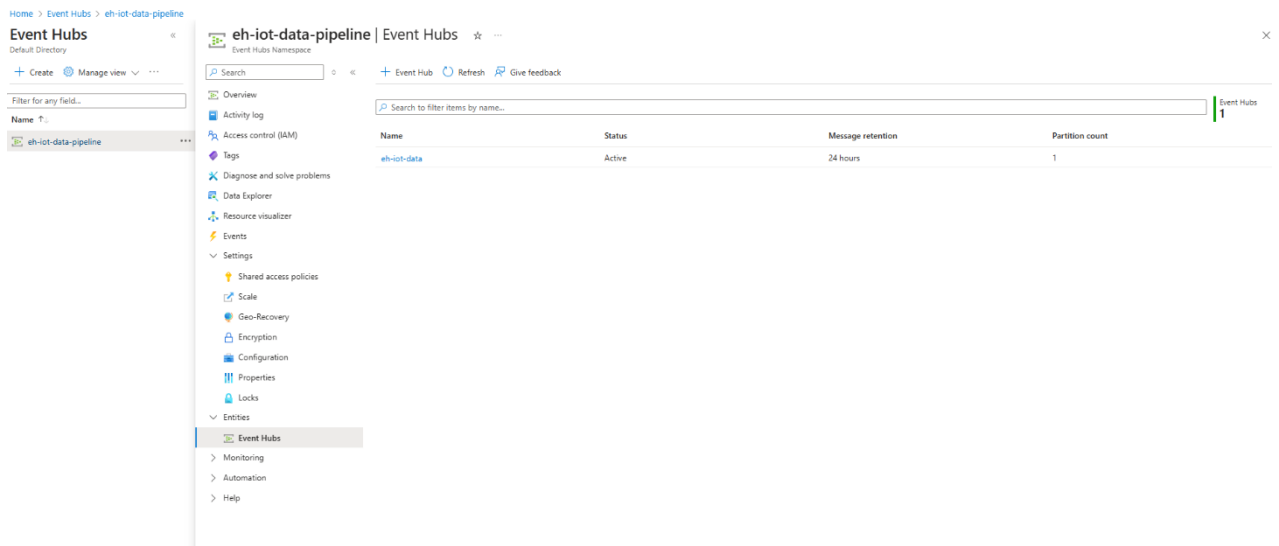
- ❖ Azure Event Hubs is a scalable, real-time data streaming service that enables the creation of real-time data ingestion pipelines, allowing consumer groups to analyse data efficiently.
- ❖ In this project Event Hubs was implemented as an intermediary between IoT Hub and Azure Databricks.
- ❖ Reasons for utilising Event Hub:
  - Event Hub is designed for high volume data streaming, providing a scalable ingestion layer when a large number of IoT devices are connected to IoT Hub.
  - Event Hub enables multiple consumers to read the data concurrently without interference.
  - Event Hub enables the direct storage of data in Azure Data Lakes (not utilised in this scenario) and allows for connectivity with various Azure services.

## Steps:

### 1. Create a Event Hub Namespace



### 2. Create a Event Hub Instance



### 3. Create a Message routing from IoT Hub to Event Hub

IoT Hub | Message routing

You have created one or more custom routes which would stop messages from flowing to the built-in endpoint. Click here to add a route to the built-in endpoint if you would like to keep the data flowing there. →

+ Add

Routes Custom endpoints Enrich messages

Create an endpoint, and then add a route (you can add up to 100 routes from each IoT Hub). Since routing is based on a matching query, a message can be sent to multiple endpoints. Messages that don't match a query are automatically sent to messages/events if you've enabled the fallback route. When you create new endpoints and routes, messages stop flowing to the built-in endpoint unless you create a separate route and direct them there. If no routes to the built-in endpoint exist, enabling a fallback route will direct any messages that don't match a route query to that endpoint. [Learn more](#)

disable fallback route

| Name                      | Data Source    | Routing Query | Endpoint           | Enabled |
|---------------------------|----------------|---------------|--------------------|---------|
| route-iot-hub-to-eventhub | DeviceMessages | true          | iothub-to-eventhub | true    |

### 4. Create a Shared Policy on Event Hub to enable Listen to the Event hub data from Databricks

Microsoft Azure | Upgrade

eh-iot-data (eh-iot-data-pipeline/eh-iot-data) | Shared access policies

Search

Policy Claims

| Policy                    | Claims |
|---------------------------|--------|
| ehdbpolicy                | Listen |
| iothubrouter_iot-hub-data | Send   |

SAS Policy: ehdbpolicy

Event Hub

Regenerate primary key Regenerate secondary key Delete

Event Hubs access control with Shared Access Signatures [Learn more](#)

Claims

☐ Manage

☐ Send

☒ Listen

Primary key

Secondary key

Primary connection string

Secondary connection string

SAS policy ARM ID

☐ Show AMQP connection strings

### 3. Data Transformation

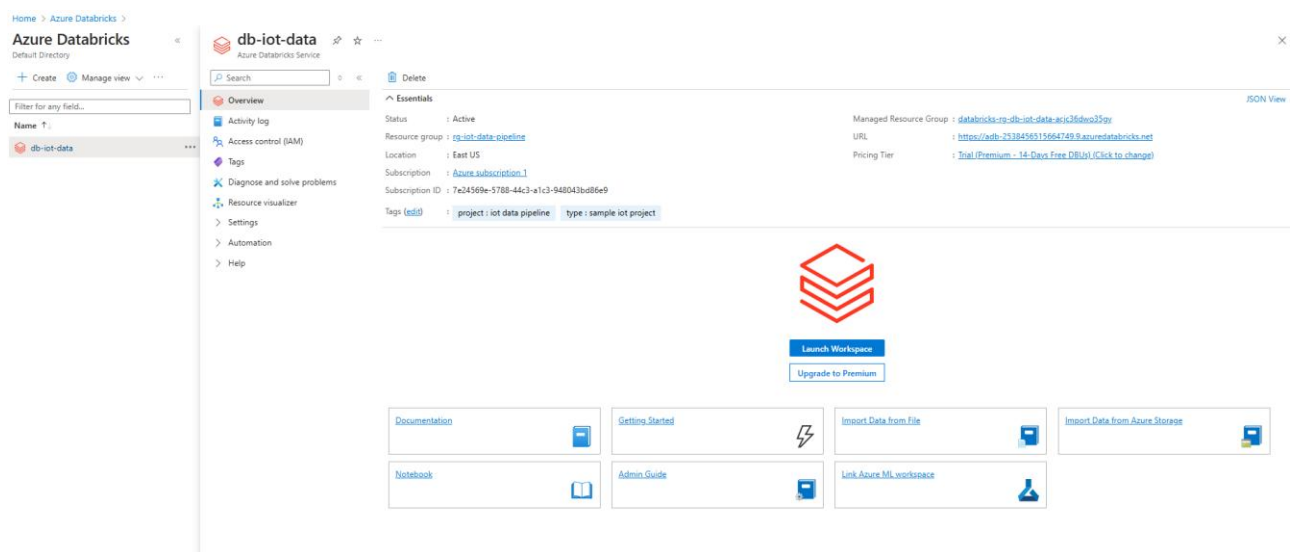
- ❖ Azure Databricks was used for data cleaning, aggregation and transformation. Data ingested from Event Hub was processed within the Databricks environment and stored in Delta Lake. The Medallion Architecture was implemented to organize and store the data facilitating efficient data analytics.

#### i) Azure Databricks

- ❖ Azure Databricks is a large scale data processing platform built on Apache Spark. It offers robust and efficient data processing capabilities, tools for building data pipelines, a Delta Lake storage layer, access control and data governance through Unity Catalog and an environment for developing machine learning models.

#### Steps:

1. Create a Databricks workspace in Premium tier to enable Unity Catalog



2. Launch the databricks workspace and create a single node cluster (To minimize the cost) with general purpose hardware (14GB and 4 cores). Set the termination time as 60 minutes with no activity reduce the cost.

Microsoft Azure

databricks

Search data, notebooks, recents, and more...CTRL + P

New

Workspace

Recents

Catalog

Workflows

Compute

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Pipelines

Machine Learning

Playground

Experiments

Features

Models

Serving

Compute > New compute > Simple form: OFF

Nuwan Galappaththi's Cluster

Policy

Unrestricted

Multi node

Single node

Access mode

Dedicated (formerly: Single user)

Single user access

Nuwan Gal...

Performance

Databricks runtime version

Runtime: 15.4 LTS (Scala 2.12, Spark 3.5.0)

Use Photon Acceleration

Node type

Standard\_DS3\_v214 GB Memory, 4 Cores

Terminate after60minutes of inactivity

Tags

Add tags

KeyValueAdd

> Automatically added tags

> Advanced options

## Cluster Summary:

Summary

1 Driver

14 GB Memory, 4 Cores

Runtime

15.4.x-scala2.12

Unity Catalog

Photon

Standard\_DS3\_v2

1.5 DBU/h

## 3. Set the working library

Nuwan Galappaththi's Cluster

ConfigurationNotebooks (0)LibrariesEvent logSpark UIDriver logsMetricsAppsSpark compute UI - Master

Filter libraries

| Status | Name                                                  | Type  | Source |
|--------|-------------------------------------------------------|-------|--------|
|        | com.microsoft.azure:azure-eventhubs-spark_2.12:2.3.22 | Maven | -      |

#### 4. Start a new notebook and attach it to the cluster

The screenshot shows the Microsoft Azure Databricks interface. A new notebook is being created with the title "Real time IoT Data Processing with Azure Databricks". The notebook content includes the following sections:

- Data Sources:** Streaming data from IoT devices.
- Ingestion:** Azure Event Hubs for capturing real-time data.
- Processing:** Azure Databricks for stream processing using Structured Streaming.
- Storage:** Processed data stored Azure Data Lake (Delta Format).
- Visualisation:** Data visualized using Power BI.

**Tools and Technologies Used**

- Raspberry Pi Azure IoT Online Simulator (<https://azure-samples.github.io/raspberry-pi-web-simulator/>)
- Azure IoT Hub
- Azure Event Hub
- Azure Databricks (Unity Catalog enabled)
- Azure Data Lake Storage Gen2
- Power BI

**Azure Databricks Configuration**

- Single Node Compute Cluster:** 15.4 LTS (includes Apache Spark 3.5.0, Scala 2.12)
- Maven Library installed on Compute Cluster:** com.microsoft.azure:azure-eventhubs-spark\_2.12:2.3.22

A cluster dropdown menu is open, showing "Nuwan Galappaththi's Cluster" as the selected cluster. The cluster details show it is a DBR 15.4 LTS cluster with 14 GB+4 Cores.

#### ii) Code explanation

##### i) Importing Libraries

```
#Importing Libraries
from pyspark.sql.functions import *
from pyspark.sql.types import *
from pyspark.sql import SparkSession
```

##### ii) Medallion data storage architecture with bronze, silver and gold schemas within 'db\_iot\_data\_2538456515664749' catalog

```
# Medallion data storage architecture with bronze, silver and gold schemas within 'db_iot_data_2538456515664749' catalog

spark = SparkSession.builder.getOrCreate()

# Define catalog and schemas
catalog_name = "db_iot_data_2538456515664749"
schemas = ["bronze", "silver", "gold"]

for schema in schemas:
    schema_full_name = f"{catalog_name}.{schema}"
    try:
        spark.sql(f"CREATE SCHEMA IF NOT EXISTS {schema_full_name}")
        print(f"Schema created or already exists: {schema_full_name}")
    except Exception as e:
        print(f"Failed to create schema '{schema_full_name}': {e}")
```

### iii) Azure Event Hub configurations

```
# Set up Azure Event Hub connection config
connectionString = "add connection string here"

eventHubName = "eh-iot-data"

ehConf = {
  'eventhubs.connectionString' : sc._jvm.org.apache.spark.eventhubs.EventHubsUtils.encrypt(connectionString),
  'eventhubs.eventHubName': eventHubName
}
```

### iv) Bronze Layer

- Read from Event Hub and write data to the bronze layer with fault tolerance

```
# Read and write data to the bronze schema
# Reading stream: Load data from Azure Event Hub into DataFrame
df = spark.readStream \
  .format("eventhubs") \
  .options(**ehConf) \
  .load()

# Visualize the data for debugging
df.display()

# Writing stream: Persist the streaming data to a Delta table 'db_iot_data_2538456515664749.bronze.weather' in 'append' mode with checkpointing
df.writeStream\
  .option("checkpointLocation", "/mnt/db_iot_data_2538456515664749/bronze/weather")\
  .outputMode("append")\
  .format("delta")\
  .toTable("db_iot_data_2538456515664749.bronze.weather")
```

### v) Silver Layer

- Defining the schema

```
# Defining the schema for the JSON object
json_schema = StructType([
  StructField("temperature", IntegerType()),
  StructField("humidity", IntegerType()),
  StructField("windSpeed", IntegerType()),
  StructField("windDirection", StringType()),
  StructField("precipitation", IntegerType()),
  StructField("conditions", StringType())
])
```



- Read from bronze layer, transform and write back to the silver layer with fault tolerance

```
# Load streaming data from the 'bronze.weather' Delta table
df = spark.readStream\
    .format("delta")\
    .table("db_iot_data_2538456515664749.bronze.weather")\
    .withColumn("body", col("body").cast("string"))\
    .withColumn("body", from_json(col("body"), json_schema))\
    .select("body.temperature", "body.humidity", "body.windSpeed", "body.windDirection", "body.precipitation", "body.conditions", col("enqueuedTime").alias('timestamp'))

# Visualize the transformed data for debugging
df.display()

# Save the transformed data to the 'silver.weather' Delta table in 'append' mode with checkpointing for data reliability
df.writeStream\
    .option("checkpointLocation", "/mnt/db_iot_data_2538456515664749/silver/weather")\
    .outputMode("append")\
    .format("delta")\
    .toTable("db_iot_data_2538456515664749.silver.weather")
```

## vi) Gold Layer

- Read from silver layer, aggregate and write back to the gold layer with fault tolerance

```
# Read from 'silver.weather' apply watermarking and calculate average weather values
df = spark.readStream\
    .format("delta")\
    .table("db_iot_data_2538456515664749.silver.weather")\
    .withWatermark("timestamp", "1 minutes") \
    .groupBy(window("timestamp", "5 minutes")) \
    .agg(avg("temperature").alias('temperature'), avg("humidity").alias('humidity'), avg("windSpeed").alias('windSpeed'), avg("precipitation").alias('precipitation'))\
    .select('window.start', 'window.end', 'temperature', 'humidity', 'windSpeed', 'precipitation')

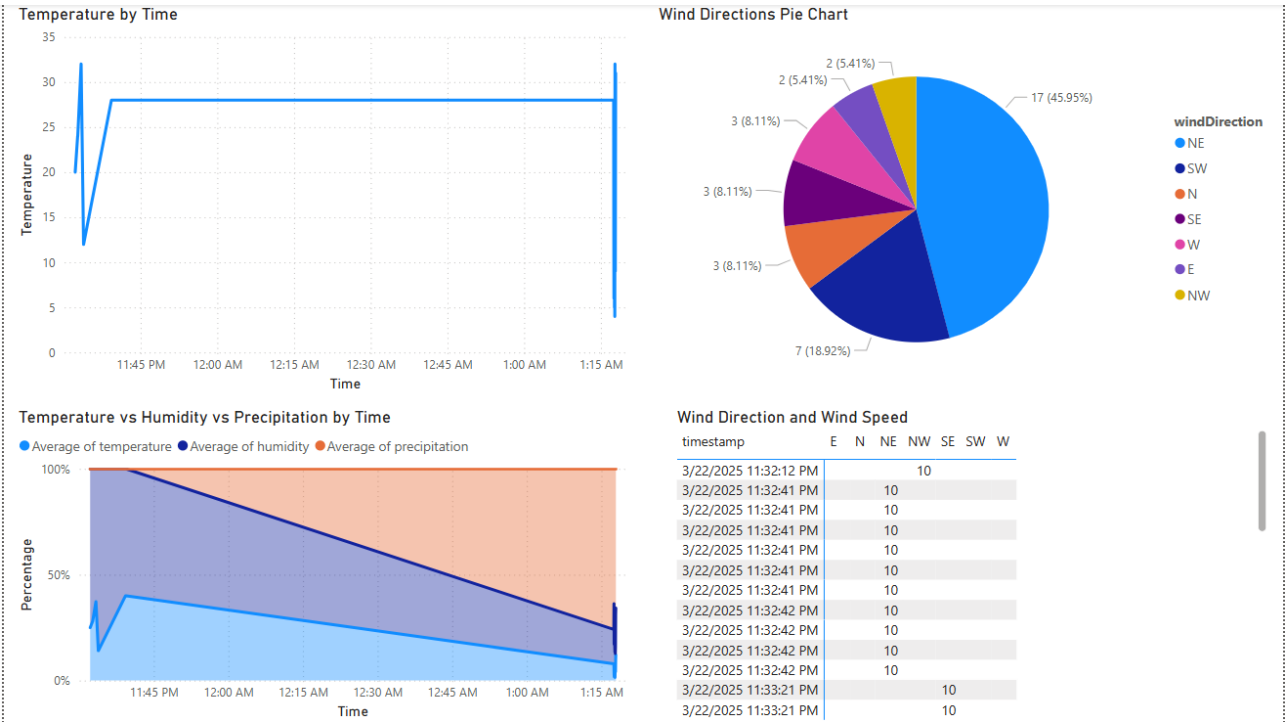
# Visualize the aggregated data for debugging
df.display()

# Save the aggregated data to the 'gold.weather_summary' Delta table in 'append' mode with checkpointing for data reliability
df.writeStream\
    .option("checkpointLocation", "/mnt/db_iot_data_2538456515664749/gold/weather_summary")\
    .outputMode("append")\
    .format("delta")\
    .toTable("db_iot_data_2538456515664749.gold.weather_summary")
```

## 4. Data Visualization

- ❖ Power BI was used for data visualization and report creation. It was directly connected to the Silver layer storage which contains non aggregated data, allowing for detailed insights. If aggregations are required, they can be implemented within Power BI.

Ex:

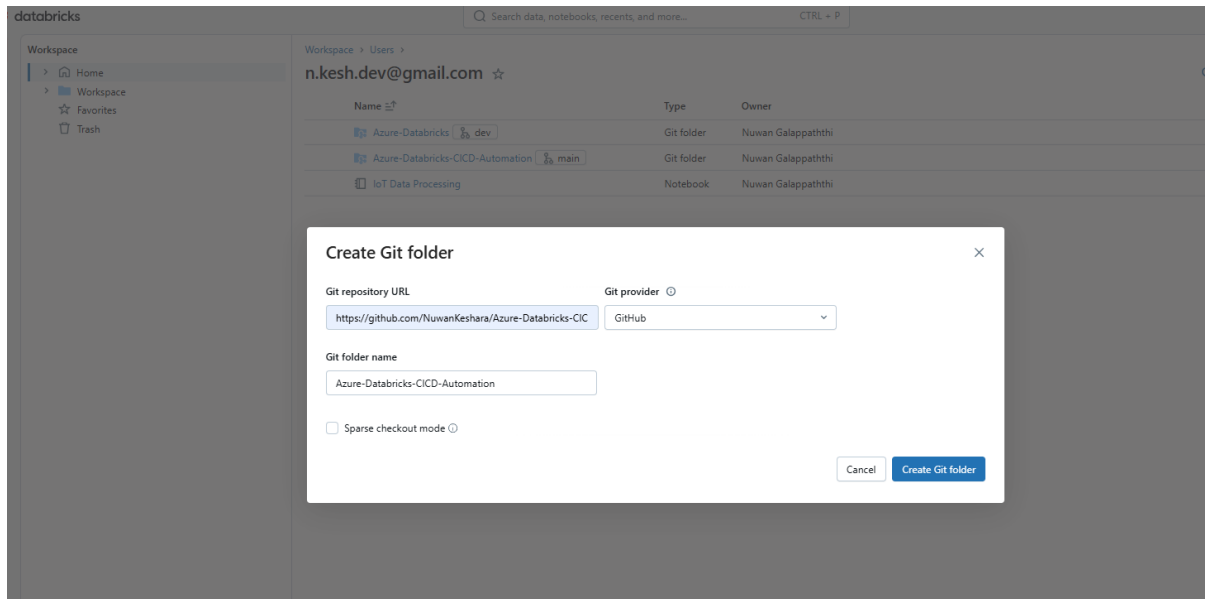


## 5. Version Control

- ❖ In addition to Azure Databricks versioning, continuous integration (CI) was implemented for the notebooks using Git. Both GitHub and Azure DevOps repositories were utilised for version control.

*Steps:*

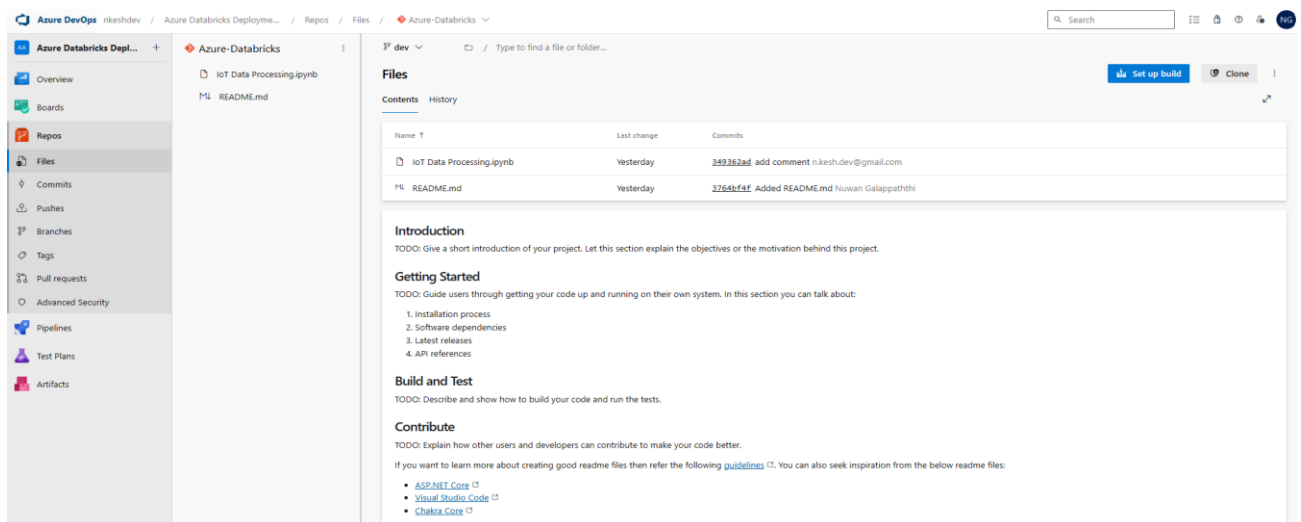
1. Create Git folder in Databricks and connect the Github (Select the GitHub as the git provider)



2. Generate a personal access token from GitHub and set it in Databricks Linked Accounts section

### i) Azure DevOps

- ❖ Same as the above create a git folder and select the git provider as Azure DevOps



## References

- [<https://azure-samples.github.io/raspberry-pi-web-simulator/>] - Microsoft
- [<https://learn.microsoft.com/en-us/azure/iot-hub/iot-hub-compare-event-hubs>] - Microsoft
- [<https://www.youtube.com/watch?v=2DiC96shSZY>] – HarshalkumarJain
- [<https://stackoverflow.com/questions/74885334/azure-iot-hub-routing-to-event-hub9>] - Stackoverflow (Jan 2022)
- [<https://learn.microsoft.com/en-us/azure/databricks/data-governance/unity-catalog/>] - Microsoft
- [<https://medium.com/@hwangdb/azure-databricks-unity-catalog-part-1-uc-concepts-and-components-2d713f4cb423>] - medium (March 2023)
- [<https://medium.com/@junshan0/medallion-architecture-what-why-and-how-ce07421ef06f>] - medium (Feb 2024)